

```

1681 ordered_choices: list[tuple[dict[str, Any], tuple[Any, ...]]] = list(zip(urgent, initial_choices))
1682 for spec in remaining:
1683     candidates = options(spec, forced_unit=spec.get("forced_unit"))
1684     if not candidates:
1685         raise RuntimeError(f"No feasible resource assignment for {spec}")
1686     ordered_choices.append((spec, min(candidates, key=lambda x: x[0])))
1687     # Commit immediately so later options see updated readiness.
1688     _, type_id, unit_id, battery_id, timeline = ordered_choices[-1][1]
1689     unit_ready[unit_id] = timeline["return_s"]
1690     battery_ready[battery_id] = timeline["return_s"] + charge_time_s(
1691         timeline["return_soc"], data["battery_inventory"][type_id]["full_charge_s"]
1692     )

```

C.3 问题三：链路判定、连续认证与固定波次执行

代码清单6实现直连或中继链路的距离、自由空间损耗、DEM 遮挡附加损耗和余量判定，是通信分段与连续认证共同调用的基础函数。

Listing 6 空地链路状态与余量计算

```

477 def fspl_db(freq_mhz: float, distance_km: float) -> float:
478     return 32.45 + 20.0 * math.log10(freq_mhz) + 20.0 * math.log10(max(distance_km, 1e-9))
479
480
481 def link_state(
482     dem: DemGrid,
483     a: tuple[float, float, float],
484     b: tuple[float, float, float],
485     threshold_db: float,
486     freq_mhz: float,
487     obstruction_db: float,
488 ) -> dict[str, Any]:
489     lon1, lat1, alt1 = a
490     lon2, lat2, alt2 = b
491     horizontal = horizontal_distance_m(lon1, lat1, lon2, lat2)
492     frac, terrain, _ = dem.path_elevations(lon1, lat1, lon2, lat2, step_m=15.0)
493     sight = alt1 + frac * (alt2 - alt1)
494     if len(frac) > 2:
495         clearance = sight[1:-1] - terrain[1:-1]
496         min_clearance = float(np.min(clearance))
497         blocked = bool(np.any(clearance <= 0.0))
498     else:
499         min_clearance = float("inf")
500         blocked = False
501     distance_3d_km = math.hypot(horizontal, alt2 - alt1) / 1000.0
502     loss = fspl_db(freq_mhz, distance_3d_km) + (obstruction_db if blocked else 0.0)
503     return {
504         "available": bool(loss <= threshold_db + 1e-12),
505         "blocked": blocked,
506         "loss_db": loss,
507         "margin_db": threshold_db - loss,
508         "min_clearance_m": min_clearance,
509         "distance_3d_km": distance_3d_km,
510     }

```

代码清单7给出单个时间区间的保守证书构造。程序把移动端点在局部平面中的轨迹包络为矩形，利用距离上界和经过 DEM 像元集合形成最坏情形链路余量下界；若该下界非负，则整个连续区间均可判为可用，而不是只依赖端点或中点采样。

Listing 7 单区间保守链路余量证书

```

709 def interval_link_certificate(
710     dem: DemGrid,
711     moving0: tuple[float, float, float],
712     moving1: tuple[float, float, float],
713     fixed: tuple[float, float, float],
714     threshold_db: float,
715     freq_mhz: float,
716     obstruction_db: float,
717 ) -> dict[str, Any]:

```